

COMPSYS302: Python

5. Input and Output

Ho Seok AHN

hs.ahn@auckland.ac.nz

5.1 Text Input and Output

□ The print Statement

- The **print** statement has the following syntax:

`print (expressions[,...])`

- **expressions** is a list of zero or more expressions separated by commas.
- After all expressions, **print** by default outputs `\n` to terminate the line.
- The **print** statement is usually used for debugging your code.

- Ex) 5_01_Print.py

```
a = 'COMPSYS'
```

```
b = 302
```

```
print (a)
```

```
print (b)
```

```
print (a), (b)
```

```
print (a, b)
```

```
print (a, type(a))
```

```
print (b, type(b))
```

```
print (str(b), type(str(b)))
```

(result)

COMPSYS

302

COMPSYS

COMPSYS 302

COMPSYS <class 'str'>

302 <class 'int'>

302 <class 'str'>

5.1 Text Input and Output

□ Alignment

- You have two methods to align printing outputs.

1. Using functions of string objects
2. Using format function of print statement

- Ex) 5_02_AlignmentString.py (square table)

```
for x in range(1,6):
```

```
    print(x, '*', x, '=', x*x)
```

```
print('\t')
```

```
for x in range(1,6):
```

```
    print(x, '*', x, '=', str(x*x).rjust(5))
```

```
print('\t')
```

```
for x in range(1,6):
```

```
    print(x, '*', x, '=', str(x*x).ljust(3))
```

```
print('\t')
```

```
for x in range(1,6):
```

```
    print(x, '*', x, '=', str(x*x).center(3))
```

```
print('\t')
```

```
for x in range(1,6):
```

```
    print(x, '*', x, '=', str(x*x).zfill(3))
```

(result)

1 * 1 = 1

2 * 2 = 4

3 * 3 = 9

4 * 4 = 16

5 * 5 = 25

1 * 1 = 1

2 * 2 = 4

3 * 3 = 9

4 * 4 = 16

5 * 5 = 25

1 * 1 = 1

2 * 2 = 4

3 * 3 = 9

4 * 4 = 16

5 * 5 = 25

1 * 1 = 1

2 * 2 = 4

3 * 3 = 9

4 * 4 = 16

5 * 5 = 25

1 * 1 = 001

2 * 2 = 004

3 * 3 = 009

4 * 4 = 016

5 * 5 = 025

5.1 Text Input and Output

□ Alignment

- You have two methods to align printing outputs.

1. Using functions of string objects
2. Using **"format()"** function of print statement

- Ex) 5_03_AlignmentFormat.py

```
print ("{0} is {1}".format("apple", "red"))  
print ("{0} is {1} or {2}".format("apple", "red", "green"))  
print ("{item} is {colour}".format(item="apple", colour="red"))
```

```
dic = {"item":"apple", "colour":"red"}  
print ("{item} is {colour}".format(**dic))
```

```
item = "apple"  
colour = "green"  
print ("{0[item]} is {0[colour]}".format(locals()))  
print ("{item} is {colour}".format(**locals()))
```

(result)

apple is red

apple is red or green

apple is red

apple is red

apple is green

apple is green

5.1 Text Input and Output

□ The input Statement

- You can use built-in function **input**.
- The input value is **string** type.
- Ex) 5_04_Input.py

```
a = input("insert any keys: ")  
print (a)  
print (type(a))
```

(result)

insert any keys: 123

123

<class 'str'>

insert any keys: apple

apple

<class 'str'>

5.2 File Input and Output

□ The open Statement

- You can create a Python file object with the built-in **open**, which has the following syntax:

`open(filename, mode)`

- **open** opens the file named by 'filename', which must be a string that denotes any path to a file.

- Ex) 5_05_FileIO.py

```
f = open('test.txt', 'w')
f.write("Hello. This message is from the file")
f.close()
f = open('test.txt', 'r')
print (f.read())
f.close()
print (f.closed)
```

(result)

Hello. This message is from the file

True

5.2 File Input and Output

□ The open Statement

- Examples of mode:
 - `'r'`
 - The file must already exist, and it is opened in **read-only** mode.
 - `'w'`
 - The file is opened in **write-only** mode.
 - The file is truncated and **overwritten** if it already exists, or created if it does not exist.
 - `'a'`
 - The file is opened in **write-only** mode.
 - The file is kept intact if it already exists, and the data you write is **appended** to what's already in the file.
 - The file is created if it does not exist.